



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorio de docencia

Laboratorio de Computación Salas A y B

Profesor(a):

M.C. René Adrián Dávila Pérez

Asignatura:

Programación orientada a objetos

Grupo:

7

No de práctica(s):

01

Integrante(s):

424045919 | 323087067 | 426060033 | 426059961 | 323297930

No de lista o brigada:

1

Semestre:

2027-1

Fecha de entrega:

28 de agosto de 2026

Observaciones:

Calificación:

Índice

1. Introducción	3
1.1. Planteamiento del problema	3
1.2. Motivación	3
1.3. Objetivos	3
2. Marco Teórico	4
2.1. Programación Estructurada vs. Conceptos de Interfaces Gráficas	4
2.2. Manejo de Entrada por Consola y Estructuras de Control	4
2.3. Entorno de Ejecución en Java y Compilador JIT	4
3. Desarrollo	4
3.1. Actividad 1: Consulta y Análisis Conceptual de la GUI Paint	4
3.2. Actividad 2: Lógica Algorítmica de la Calculadora CLI	5
3.3. Casos de Prueba de Entrada y Salida	10
4. Resultados	10
4.1. Respuesta y Propuesta Generada por el LLM	10
4.2. Evidencias de Ejecución en Terminal	11
5. Conclusiones	12
6. Referencias	12

1. Introducción

1.1. Planteamiento del problema

En estas primeras semanas del curso es fundamental comprender los pilares del lenguaje Java y cómo abordar la transición entre diferentes paradigmas de programación. En esta práctica se plantearon dos actividades. La primera consistió en realizar un ejercicio de interacción con un Modelo de Lenguaje Grande (LLM) para consultar cómo se implementaría en Java la interfaz gráfica (GUI) de una aplicación de dibujo (Paint) mostrada en clase, analizando conceptualmente los componentes requeridos. La segunda actividad consistió en la implementación práctica de una calculadora por línea de comandos (CLI) bajo el paradigma estructurado, capaz de procesar operaciones aritméticas básicas y controlar errores de ejecución, como la división entre cero y el cálculo de raíces cuadradas de números negativos.

1.2. Motivación

El uso de modelos de inteligencia artificial como herramientas de apoyo académico permite a los estudiantes de ingeniería explorar arquitecturas complejas de software antes de haber cubierto todos los temas avanzados del temario. Analizar la respuesta de una IA sobre componentes gráficos, a la par de desarrollar código estructurado en consola, ayuda a reforzar el uso de variables, ciclos y condicionales, al mismo tiempo que ofrece una visión general de los temas de programación orientada a objetos que se abordarán posteriormente durante el semestre.

1.3. Objetivos

Objetivo general:

Identificar y probar el entorno de ejecución y el lenguaje de programación orientado a objetos que se utilizará durante el curso.

Objetivos específicos:

- Analizar la respuesta generada por un LLM respecto a la selección de componentes Swing/AWT (JComboBox, JCheckBox, JButton y JPanel) necesarios para reproducir una interfaz tipo Paint.
- Construir un menú interactivo en consola haciendo uso de las estructuras condicionales `switch` y de un ciclo `do-while` para el manejo de operaciones aritméticas y errores matemáticos.
- Explicar la arquitectura de ejecución de Java, detallando la función del compilador JIT (Just-In-Time) dentro de la Java Virtual Machine (JVM).

2. Marco Teórico

2.1. Programación Estructurada vs. Conceptos de Interfaces Gráficas

El paradigma estructurado organiza el código de forma secuencial utilizando estructuras de control como selección e iteración.¹ Por otro lado, las interfaces gráficas de usuario (GUI) operan bajo un paradigma orientado a eventos, en el que el programa reacciona a las acciones del usuario sobre componentes visuales.² La biblioteca Swing de Java proporciona elementos como `JComboBox` para selecciones desplegables, `JCheckBox` para opciones booleanas, `JButton` para acciones puntuales y `JPanel` para áreas de renderizado.³

2.2. Manejo de Entrada por Consola y Estructuras de Control

Para la captura de datos desde la terminal se emplea la clase `Scanner` del paquete `java.util`, la cual analiza el flujo de entrada estándar (`System.in`).⁴ La interacción con el usuario se mantiene mediante un ciclo `do-while`, que despliega el menú hasta que se selecciona la opción de salida, mientras que la elección se evalúa mediante una estructura `switch`.

2.3. Entorno de Ejecución en Java y Compilador JIT

Al compilar un programa Java con `javac`, se genera un código intermedio denominado bytecode.⁵ Este código es ejecutado por la Java Virtual Machine (JVM), que utiliza interpretación y, para mejorar el rendimiento, integra un compilador JIT (Just-In-Time). El JIT monitorea la ejecución en tiempo real para detectar bloques de código recurrentes o bucles. Al identificar un *hot spot*, el compilador traduce ese bytecode a código máquina nativo del procesador y lo conserva en caché, permitiendo que las siguientes ejecuciones de esas instrucciones sean más eficientes.⁵

3. Desarrollo

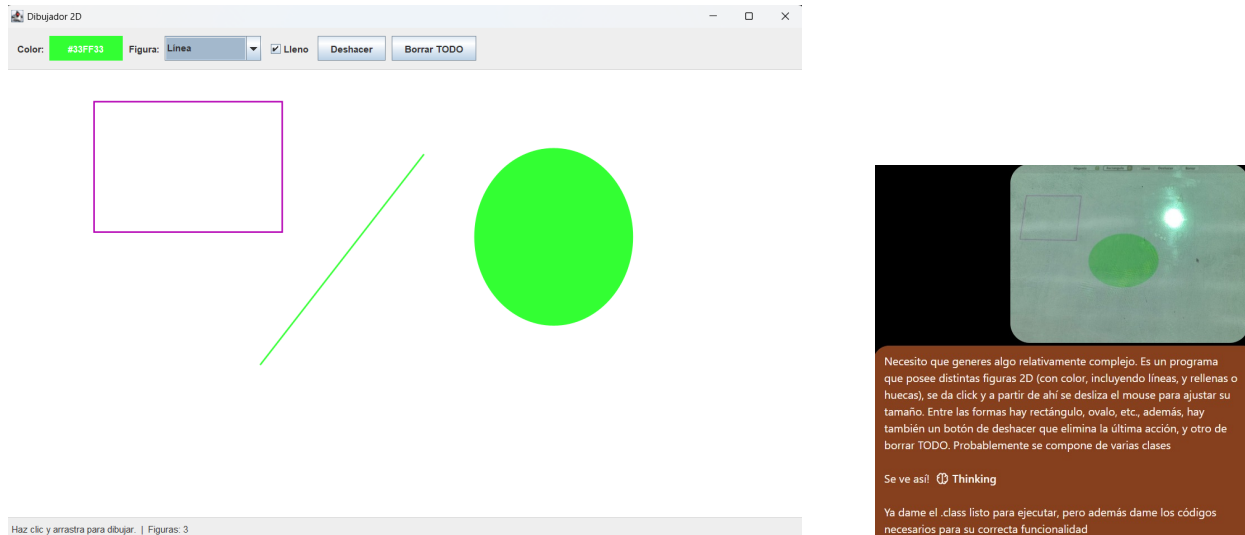
3.1. Actividad 1: Consulta y Análisis Conceptual de la GUI Paint

En esta actividad no se requirió programar la interfaz gráfica. En su lugar, se utilizó un LLM como herramienta de investigación para analizar la imagen presentada por el profesor.

1. **Formulación de la consulta:** Se proporcionó la captura de la interfaz de dibujo al LLM para solicitar una explicación de cómo construir esa GUI en Java.
2. **Identificación de componentes:** Se analizó la sugerencia del LLM, que identificó el uso de una ventana principal, un panel superior de controles que aloja dos `JComboBox`

para colores y formas, un `JCheckBox` para la opción de relleno y botones `JButton` para deshacer y borrar.

3. **Lógica de dibujo propuesta:** El LLM explicó que la zona de dibujo requeriría un `JPanel` personalizado que escuche eventos del ratón para capturar coordenadas y almacenar las formas geométricas en una lista antes de redibujarlas.



Resultado de la ejecución del archivo .jar proporcionado por el LLM.

Prompt proporcionado al LLM para solicitar la generación del programa GUI Paint.

Figura 1: Evidencias de la interfaz de dibujo (Paint) utilizada en la actividad.

3.2. Actividad 2: Lógica Algorítmica de la Calculadora CLI

El programa de la calculadora por consola se estructuró en Java de la siguiente manera:

1. Se instanció un objeto de la clase `Scanner` para la lectura de datos por teclado.
2. Se implementó un ciclo `do-while` que despliega el menú de ocho opciones y mantiene en ejecución la aplicación.
3. Se integró una estructura `switch`:
 - Suma, resta, producto y potencia: primero leen dos números y realizan la operación correspondiente. La potencia utiliza `Math.pow()`.
 - División y módulo: se realiza una validación con `if` para comprobar que el divisor no sea cero. Si lo es, se despliega un mensaje de advertencia; de lo contrario, se calcula el resultado.
 - Raíz cuadrada: se valida con `if` que el número ingresado no sea negativo antes de llamar a `Math.sqrt()`.

- Salida: se termina el ciclo y se cierra el lector de teclado.

Diagramas de flujo de la solución

Como evidencia de la lógica algorítmica desarrollada, se incluyen los diagramas de flujo del menú principal y de cada operación implementada.

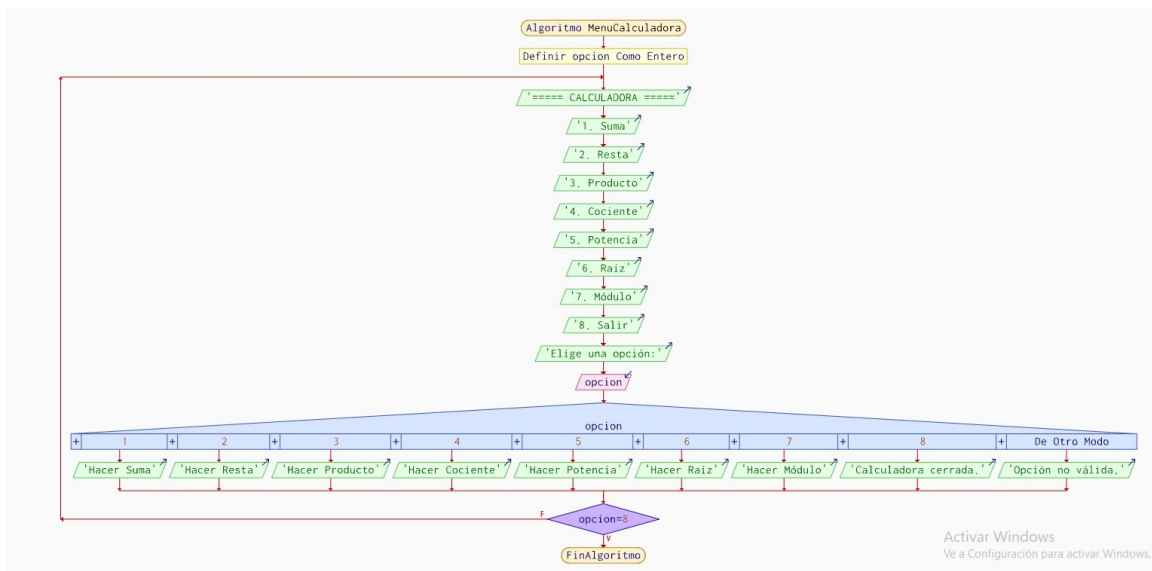


Figura 2: Diagrama de flujo del menú principal de la calculadora.

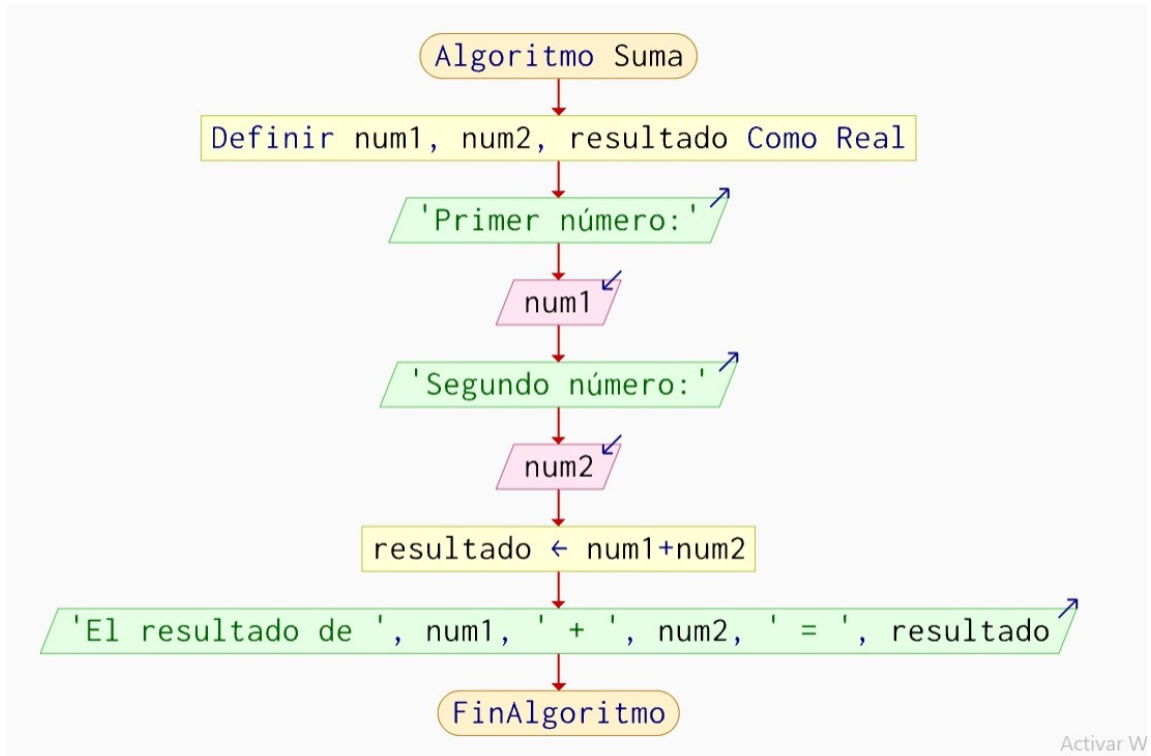


Figura 3: Diagrama de flujo de la operación de suma.

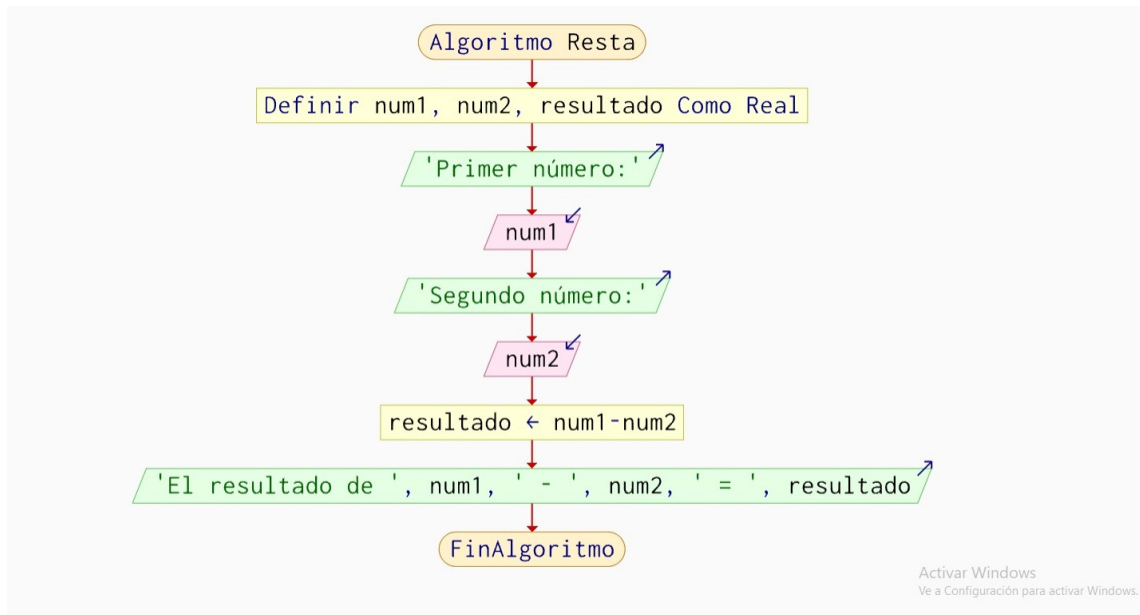


Figura 4: Diagrama de flujo de la operación de resta.

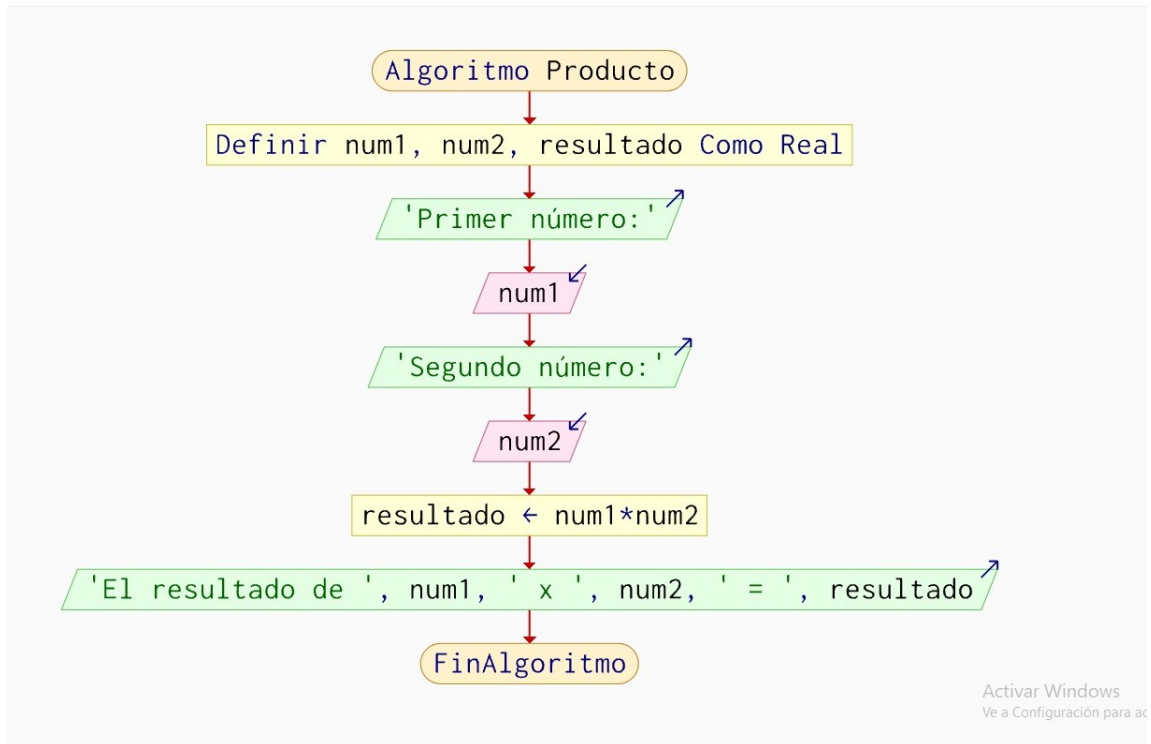


Figura 5: Diagrama de flujo de la operación de producto.

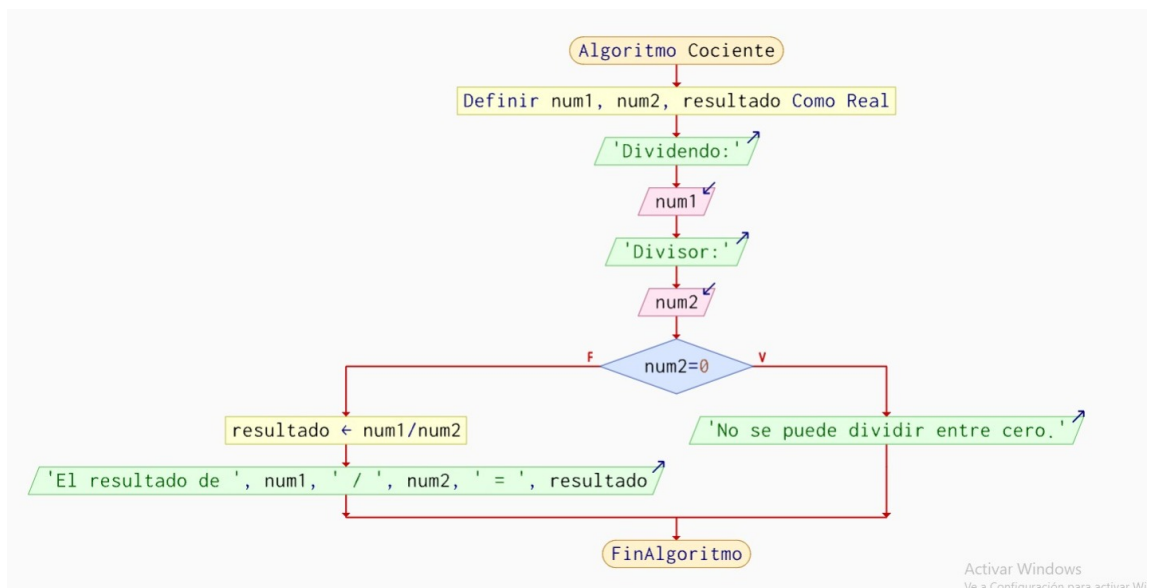


Figura 6: Diagrama de flujo de la operación de cociente.

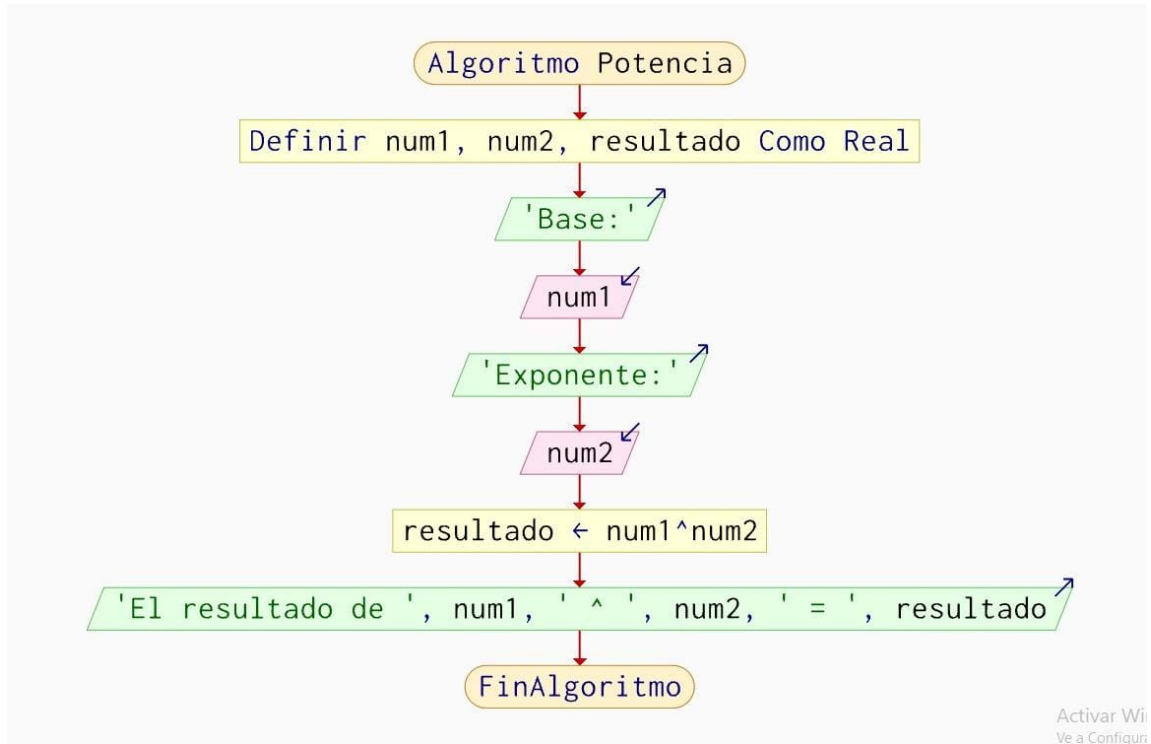


Figura 7: Diagrama de flujo de la operación de potencia.

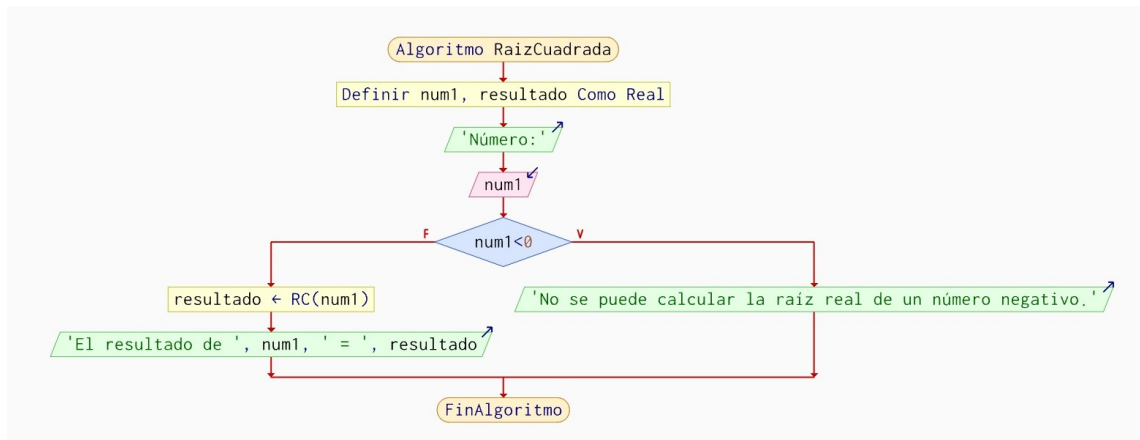


Figura 8: Diagrama de flujo de la operación de raíz cuadrada.

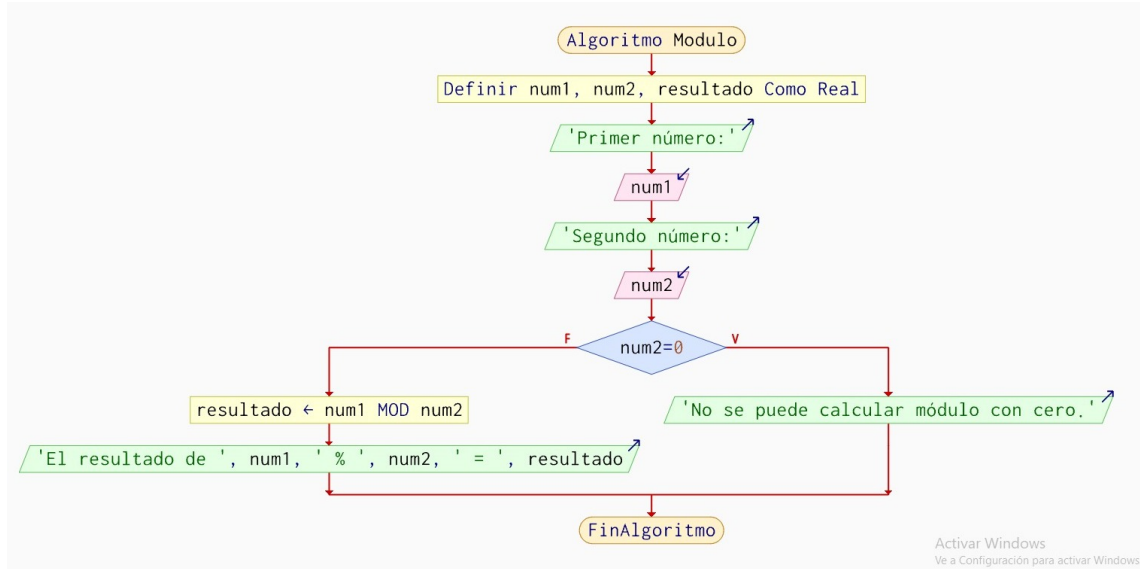


Figura 9: Diagrama de flujo de la operación de módulo.

3.3. Casos de Prueba de Entrada y Salida

Se plantearon las siguientes pruebas en consola para la calculadora:

- **Prueba 1 - Potencia:** Opción 5, base = 2.0, exponente = 3.0. Resultado: 8.0.
- **Prueba 2 - División entre cero:** Opción 4, dividendo = 10.0, divisor = 0.0. Resultado: “No se puede dividir entre cero.”
- **Prueba 3 - Raíz de negativo:** Opción 6, número = -9.0. Resultado: “No se puede calcular la raíz real de un número negativo.”

4. Resultados

4.1. Respuesta y Propuesta Generada por el LLM

A continuación se resume la propuesta conceptual obtenida tras consultar al LLM sobre cómo implementar la GUI de Paint en Java utilizando Swing:

- **Componentes de control:** se sugiere usar `JComboBox` para seleccionar el color y la forma, un `JCheckBox` para el estado del relleno y dos `JButton` para deshacer y borrar.
- **Estructura de la ventana:** organización mediante un `BorderLayout`, donde la barra de herramientas se coloca en `NORTH` y la zona de dibujo en `CENTER`.
- **Manejo de eventos:** uso de `MouseListener` para registrar las coordenadas de los clics y de un método personalizado de pintado sobre el `JPanel`.

4.2. Evidencias de Ejecución en Terminal

Se anexan las capturas de la terminal mostrando las pruebas de la calculadora y los mensajes asociados a las operaciones normales y a las validaciones implementadas.

```
PS C:\Users\Gabo\Desktop\Programacion\P00> cd P00-E1-T-
PS C:\Users\Gabo\Desktop\Programacion\P00\P00-E1-T-> cd P1
PS C:\Users\Gabo\Desktop\Programacion\P00\P00-E1-T-\P1> javac Calculadora.java
PS C:\Users\Gabo\Desktop\Programacion\P00\P00-E1-T-\P1> java Calculadora

===== CALCULADORA =====
1. Suma
2. Resta
3. Producto
4. Cociente
5. Potencia
6. Raíz
7. Módulo
8. Salir
Elige una opción: 5
Base: 2
Exponente: 5
El resultado de 2.0 ^ 5.0 = 32.0

===== CALCULADORA =====
1. Suma
2. Resta
3. Producto
4. Cociente
5. Potencia
6. Raíz
7. Módulo
8. Salir
Elige una opción: 8
Calculadora cerrada.
PS C:\Users\Gabo\Desktop\Programacion\P00\P00-E1-T-\P1> java Calculadora

===== CALCULADORA =====
1. Suma
2. Resta
3. Producto
4. Cociente
5. Potencia
6. Raíz
7. Módulo
8. Salir
Elige una opción: 4
Dividendo: 5
Divisor: 0
No se puede dividir entre cero.

===== CALCULADORA =====
1. Suma
2. Resta
```

Figura 10: Evidencia de ejecución de la calculadora en la terminal, incluyendo pruebas de operación y manejo de errores.

5. Conclusiones

Se alcanzaron satisfactoriamente los objetivos propuestos para esta práctica.

En relación con los objetivos específicos:

1. La consulta realizada al LLM permitió comprender conceptualmente la estructura de componentes Swing requeridos para crear una interfaz gráfica de dibujo, identificando la función de elementos como `JComboBox`, `JCheckBox`, `JBButton` y el manejo de eventos de ratón.
2. Se implementó con éxito la calculadora por consola en Java mediante ciclos `do-while` y estructuras `switch`, aplicando validaciones condicionales que evitan errores en tiempo de ejecución por divisiones entre cero o raíces de números negativos.
3. Se comprendió el proceso de ejecución en Java y la función del compilador JIT dentro de la JVM, entendiendo cómo optimiza el código mediante la traducción de las secciones más ejecutadas de bytecode a código máquina nativo.

Esta práctica permitió combinar la investigación de conceptos de Java mediante herramientas de IA con la práctica directa de programación estructurada en consola.

6. Referencias

- ¹ B. Téllez, “Paradigma Estructurado #4 | Fundamentos de Programación,” YouTube, 04-jul-2023. [En línea]. Disponible en: <https://www.youtube.com/watch?v=8A95ajXTRa4>
- ² Oracle, “Creating a GUI With JFC/Swing,” Oracle Java Tutorials, 2021. [En línea]. Disponible en: <https://docs.oracle.com/javase/tutorial/uiswing/>
- ³ GeeksforGeeks, “Java Swing - JComboBox with Examples,” GeeksforGeeks Tech Portal, 2024. [En línea]. Disponible en: <https://www.geeksforgeeks.org/java/java-swing-jcombobox-examples/>
- ⁴ W3Schools, “Java User Input (Scanner),” W3Schools Online Web Tutorials, 2024. [En línea]. Disponible en: https://www.w3schools.com/java/java_user_input.asp
- ⁵ IBM, “El compilador JIT,” IBM Documentation, 2023. [En línea]. Disponible en: <https://www.ibm.com/docs/es/sdk-java-technology/8?topic=reference-jit-compiler>